Lab 4 Report

Team Hotel

4/20/2025

Electronics Design Lab: ECEN 2270

Scott Lee **Tre Cade** **Henry Matar** **Ben Finneseth**

Introduction/Objectives

In lab 4, we extended our robot's motion control to the other side, copying our dual speed sensor circuit using 555 timers. Then we added the Arduino Nano Every to our circuitry in order to utilize it's abilities to control the robot through software. We used the Arduino's PWM outputs to generate stable speed-control voltages, then tuned and verified left/right motor speeds and repeatability through programmed move and turn routines. We then implemented software interrupts to count pulses for closed-loop position control. Finally, we combined these elements into a position-control system that ramps motor speeds and direction for precise linear and rotational movements, demonstrating sub-inch accuracy on tests.

Objectives

1. Construct, test, and implement second side of circuit
2. Implement and test Arduino Nano Every intro breadboard circuit
3. Test and analyze waveform outputs from Arduino Nano Every
4. Write software using polling, then interrupts, to control the robot
5. Test repeatability of code using pre-programmed routes

Goals

1. Utilize prior understanding of the circuit to build a second side of the circuit that is equivalent to the first
2. Understand the Arduino Nano Every in order to implement it into the circuit design
3. Understand the software for the Arduino Nano Every, understand the differences between polling and interrupts, as well as how to implement interrupts.

Methods and Results

Equipment List:

- SIMetrix software – SIMetrix Technologies Version 9.1
- Breadboard, jumper wires, capacitors, resistors - Generic
- AD3 WaveForms software – Digilent Version 2.23.4
- AD3 Waveform Generator bundle with flywire assembly – Digilent Part # 471-060

- BNC Adapter – Digilent Part # 410-263
- Oscilloscope probes – Digilent Part # 460-004
- Personal Computers – Various, Mac, Dell, Hp
- 18650 Batteries and barrel jack adaptor – Generic
- TLV272 OpAmp - Texas Instruments Part # TLV2721
- LMC555 CMOS Timer – Texas Instruments Part# LMC 555
- MJE200G BJT - DigiKey part #MJE200GOS-ND
- MJE210G BJT - DigiKey part # MJE210GOS-ND
- 1N4148 Small Signal Diode - ONSEMI part #1N4148
- MOSFET ZVN2106A- DigiKey part #ZVP2106GTR-ND
- MOSFET ZVP2106A - DigiKey part #ZVP2106A-ND
- PerfBoard Printed Circuit Board
- 1N5818 Small Signal Diode

4.A.1 Prelab

Lab Exploration Topic

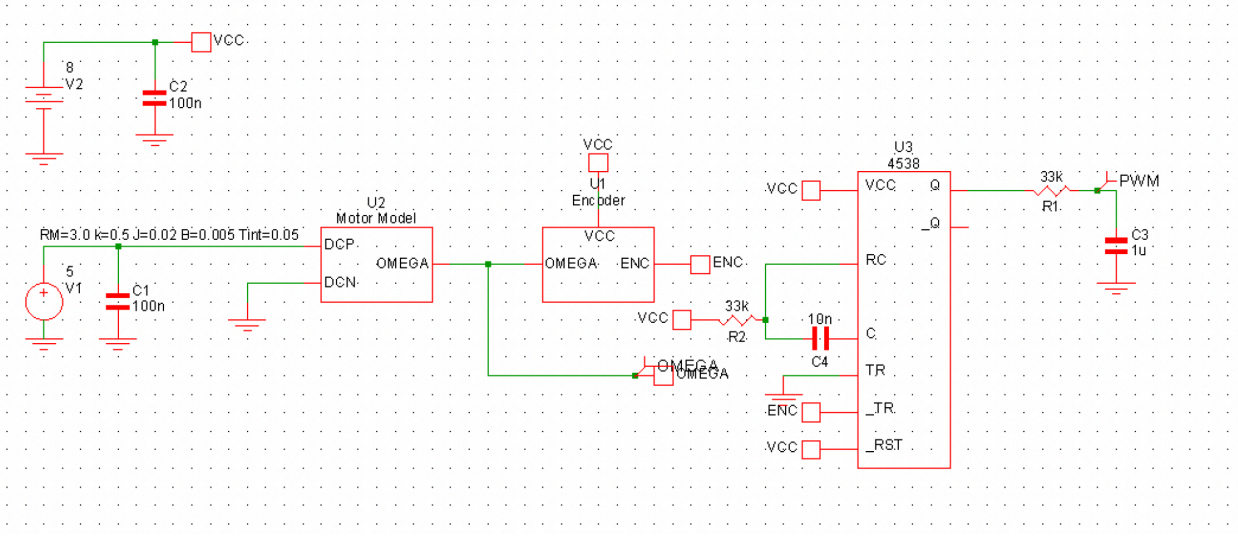
1. What are hardware and software interrupts and how do they differ? Hardware interrupts are triggered by external or peripherals, asserting an interrupt line. Software interrupts are invoked by executing a special instruction or by fault conditions. Hardware interrupts signal external events independent of code flow, whereas software interrupts are generated by the CPU's own instructions within the program.
2. What is polling? Polling is the process of having your main program loop periodically to check if a condition has been met rather than relying on an interrupt.
3. How does an interrupt work? A peripheral assets its interrupt request line, the CPU finishes the current instruction, then automatically goes to the interrupt table to fetch the ISR address. The CPU pushes the return address and status register onto the stack, disables further interrupts, jumps to the ISR, the ISR executes handling code, and an explicit return from interrupt instruction pops the status and return addresses, reenables interrupts, and resumes the interrupted code.
4. What happens in the microcontroller program execution when an interrupt occurs? Program counter and status register are pushed onto the stack, interrupts are masked to prevent nesting, the CPU loads the ISR's start address from the table, the interrupt routine runs to completion, and then the Program counter and status register are popped, interrupts are reenabled, and program execution continues.
5. What is the interrupt structure of the Arduino Nano Every? The Nano Every uses the Atmega4809 MCU, which has external interrupts, pin change interrupts. Timer interrupts,

and peripheral interrupts. You use attachInterrupt(digitalPinToInterrupt(pin), ISR, mode) to configure the interrupt.

6. Write a simple Arduino program that demonstrates the use of interrupts.

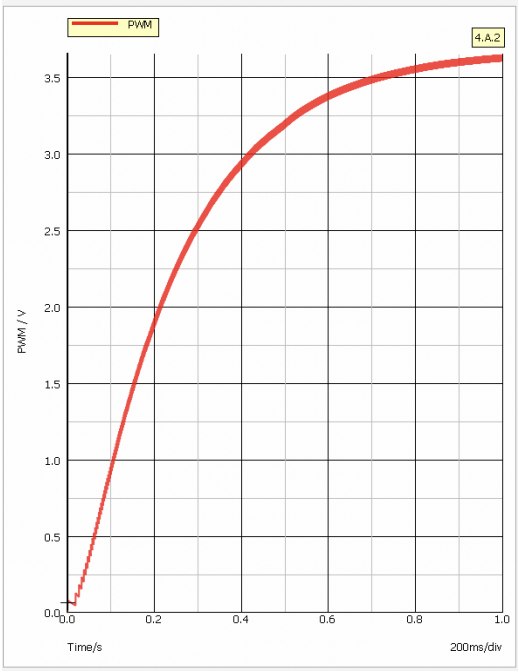
```
volatile bool ledState = LOW;

// Interrupt Service Routine toggles the LED
void blinkISR() {
  ledState = !ledState;
  digitalWrite(LED_BUILTIN, ledState);
}

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
  pinMode(2, INPUT_PULLUP);
  // Fire blinkISR on FALLING edge of pin 2:
  attachInterrupt(digitalPinToInterrupt(2), blinkISR,
FALLING);
}

void loop() {
  // main loop does nothing — LED is driven entirely by the
ISR
}
```

4.A.2 Dual Speed Sensor



Implementing a dual speed sensor, we needed to recalculate RC values to match the pulse period of the 555-based speed sensor. Using the suggested capacitance of 10uF, we decided on using

33k ohm resistors to create a pulse period of 330us. Using SIMetrix, we simulated the dual sensor circuit integrated into the robot to ensure our RC values deliver the correct output. The transient simulation measured the PWM of the circuit. Our desired result was for it to level out at 4V in which it got extremely close.



4.A.3.

To ensure the Arduino Nano Every was working we ran a simple program making the LED on the arduino blink. In order to properly connect the arduino we had to download the arduino MegaAVR boards library to connect the Nano.

```
void setup() {
 // initialize digital pin LED_BUILTIN as an output.
 pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
 digitalWrite(LED_BUILTIN, HIGH);  // turn the LED on (HIGH is the voltage level)
 delay(2000);                      // wait for a second
 digitalWrite(LED_BUILTIN, LOW);   // turn the LED off by making the voltage LOW
 delay(2000);                      // wait for a second
}
```

4.A.4

We wired the VIN pin to Vcc through a 1N5818 diode, and grounded the Arduino through pin 4, this process ensures the Arduino won't power the robot circuitry when either the battery is off and it is connected via USB.

4.A.5

The pulse width modulation frequency of the Arduino Nano Every is set to 976 hz. We want to convert this frequency PWM to an analog signal using a LPF. In short, we want to attenuate the high frequency components so that we are left with only the DC component of the fourier spectrum. Using the formula Fc=1/(2piRC), and a 10k Ohm resistor and 1uF capacitor gives a cutoff frequency of 15.9 Hz, far above the frequency of the PWM, blocking out its harmonics well. The 976 hz spectrum component is attenuated by a factor of over 60. We also wanted a time constant that was not too high, so that the response to the PWM would be fast enough to track the average duty cycle well. The high value of the resistor was also chosen so that enough power would be dissipated to not damage further parts of the circuit.

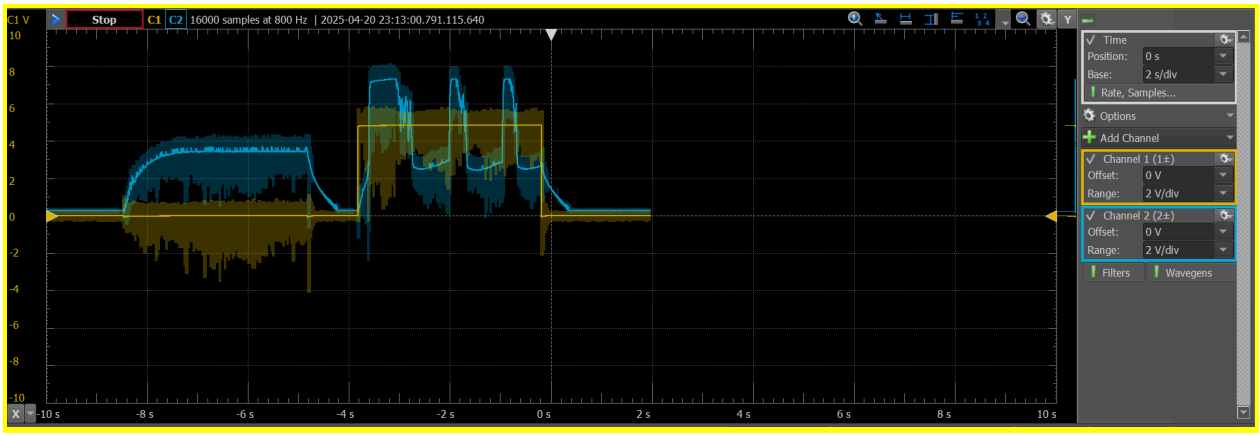
4.A.6

To test that the motors ran at the same speed for each reference voltage, we probed both frequencies of the left and right encoders. The encoder frequencies were very similar to one another. Although there was some variance in the measurement in the end, the robot drove straight. When one side of the motor was lagging behind the other, and thus turning in that direction, we lowered the resistance value of R2, and ended up settling on a 2.2k ohm value that worked the best at all voltages for the straightest drive path

4.A.7

Add open-drain MOSFET stages on D7/D8 (right) and D11/D12(left) to let the Arduino switch motor direction.

4.A.8



Probe Figure: In the top figure, channel 1 (orange) measures the pwm output of the controller for the forward direction left side of the robot, and channel 2 (blue) measures the output of the left speed sensor, both during one complete cycle of the code. Channel 1 is zero initially as the left side of the wheels on the left side move backwards. After switching directions, the pwm outputs an expected approximate 4V constant. The speed sensor dips as it is stopping/ winding down and thereafter is being compensated.

Code: The following code waits for a button input, then turns the robot counter clockwise 360 degrees, by moving the left side of the wheels backwards while the right goes forwards, pausing, and then switching the direction of each side to rotate clockwise 360 degrees, with the goal of finishing in the same place it started.

```
const int pinON = 6;     // D6 is on/off switch, active LOW
const int pinLeftForward = 11;     // connect D11 to FWD
const int pinLeftBackward = 12;  // connect D12 to BACK
const int pinLeftPWM = 10;  // connect D10 to left Vref via Left RC LPF

const int pinRightForward = 7;     // connect D11 to FWD
const int pinRightBackward = 8;  // connect D12 to BACK
const int pinRightPWM = 9;  // connect D10 to left Vref via Left RC LPF

// setup pins and initial values
void setup() {
   pinMode(pinON, INPUT_PULLUP);
   pinMode(pinLeftForward, OUTPUT);
   pinMode(pinLeftBackward, OUTPUT);
   pinMode(pinLeftPWM, OUTPUT);

   pinMode(pinON, INPUT_PULLUP);
   pinMode(pinRightForward, OUTPUT);
   pinMode(pinRightBackward, OUTPUT);
   pinMode(pinRightPWM, OUTPUT);


   pinMode(13, OUTPUT);     // on-board LED
   digitalWrite(pinLeftForward, LOW);   // stop forward
   digitalWrite(pinLeftBackward, LOW);  // stop backward
   analogWrite(pinLeftPWM, 200);  // set Vref, duty-cycle 200/255

   digitalWrite(pinRightForward, LOW);   // stop forward
   digitalWrite(pinRightForward, LOW);  // stop backward
   analogWrite(pinRightPWM, 200);  // set Vref, duty-cycle 200/255
```

```
}

void loop() {
   digitalWrite(13, LOW);    // turn LED off
   do {} while (digitalRead(pinON) == HIGH);  // wait for ON switch
   digitalWrite(13, HIGH);    // turn LED on
   delay(1000);    // wait 1 second


   digitalWrite(pinLeftForward, HIGH);  // go clockwise
   digitalWrite(pinRightBackward, HIGH);  // go clockwise
   delay(3000);    // for 3 seconds
   digitalWrite(pinLeftForward, LOW);  // go clockwise
   digitalWrite(pinRightBackward, LOW);  // go clockwise

   digitalWrite(pinRightForward, HIGH);  // go clockwise
   digitalWrite(pinLeftBackward, HIGH);  // go clockwise
   delay(3000);    // for 3 seconds
   digitalWrite(pinRightForward, LOW);  // go clockwise
   digitalWrite(pinLeftBackward, LOW);  // go clockwise

   // digitalWrite(pinRightForward, LOW);  // go clockwise
   // digitalWrite(pinLeftBackward, LOW);  // go clockwise

}
```

4.A.9 Movement Repeatability

Use your speed-control sketch to execute 2 foot forward, 180 degree turn, 2 ft forward, 180 degree turn, and measure how consistently the robot returns to start.

```
const int pinON = 6; // ON/OFF Switch
const int pinLeftForward = 11; // LEFT FORWARDS
const int pinLeftBackward = 12; //LEFT BACKWARDS
const int pinLeftPWM = 10; // LEFT PWM
const int pinRightForward = 7; // RIGHT FORWARD
const int pinRightBackward = 8; // RIGHT BACKWARD
const int pinRightPWM = 9; //RIGHT PWM

void setup() {
 pinMode(pinON, INPUT_PULLUP);
 pinMode(pinLeftForward, OUTPUT);
 pinMode(pinLeftBackward, OUTPUT);
 pinMode(pinLeftPWM, OUTPUT);
```

```
 pinMode(pinRightForward, OUTPUT);
 pinMode(pinRightBackward, OUTPUT);
 pinMode(pinRightPWM, OUTPUT);

 pinMode(13, OUTPUT); //LED
 digitalWrite(pinRightForward, LOW); // Start forward off
 digitalWrite(pinRightBackward, LOW); //Start backwards off
 analogWrite(pinRightPWM, 200); //Duty-cycle 200/255

 digitalWrite(pinLeftForward, LOW); // Start forward off
 digitalWrite(pinLeftBackward, LOW); //Start backwards off
 analogWrite(pinLeftPWM, 200); // Duty-cycle 200/255
}

void loop() {
 digitalWrite(13, LOW);
 do {} while (digitalRead(pinON) == HIGH); // Wait for button to be pressed
 digitalWrite(13, HIGH); //LED ON

 //forward 2 feet
 delay(1000); //Wait 1 second
 digitalWrite(pinLeftForward, HIGH);
 digitalWrite(pinRightForward, HIGH);
 delay(1000); //Wait 1 second

 //STOP
 digitalWrite(pinLeftForward, LOW);
 digitalWrite(pinRightForward, LOW);

 //Clockwise
 delay(1000); //Wait 1 second
 digitalWrite(pinLeftForward, HIGH); //Clockwise
 digitalWrite(pinRightBackward, HIGH); //Clockwise
 delay(1000); //Wait 1 second

 //STOP
 digitalWrite(pinLeftForward, LOW);
 digitalWrite(pinRightBackward, LOW);

 //Forward 2 feet
 delay(1000); //Wait 1 second
```

```
 digitalWrite(pinLeftForward, HIGH);
 digitalWrite(pinRightForward, HIGH);
 delay(1000); //Wait 1 second

 //STOP
 digitalWrite(pinLeftForward, LOW);
 digitalWrite(pinRightForward, LOW);

 //Counter Clockwise
 delay(1000); //Wait 1 second
 digitalWrite(pinRightForward, HIGH);
 digitalWrite(pinLeftBackward, HIGH);
 delay(1000); //Wait 1 second

 //STOP
 digitalWrite(pinRightForward, LOW);
 digitalWrite(pinLeftBackward, LOW);
}
```

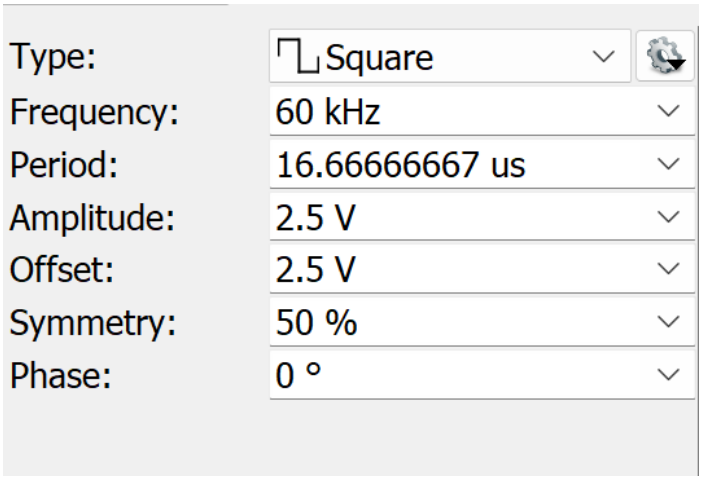
For each of our robots, we each had to alter the value of targetPulsesForward and targetPulsesTurn in order to adjust for the accuracy of our robots, but we were eventually all able to get within +- 2 inches of our beginning location after adjusting those values slightly for each person.

4.B.1 Prelab

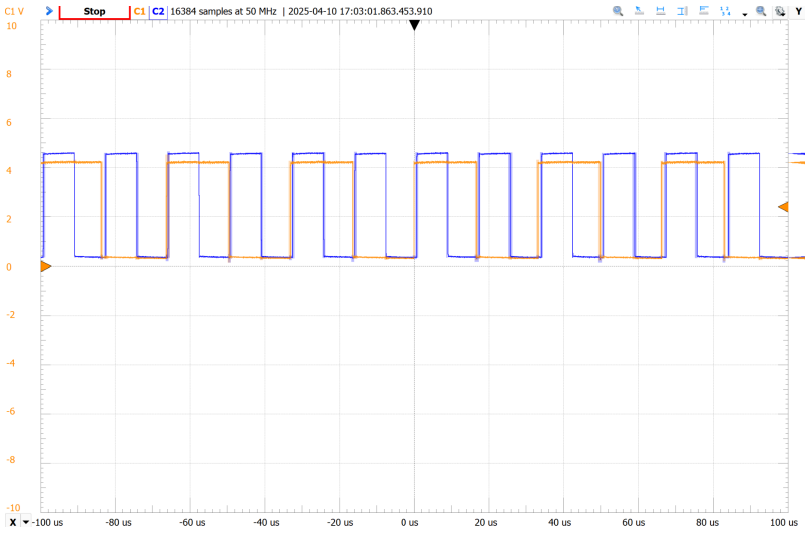
Prelab Exploration Topic - None

4.B.2 Estimate Interrupt Overhead

In order to estimate the overhead of the interrupt service routine (ISR), we uploaded an Arduino program that toggled a boolean variable inside the ISR triggered by the rising edge of pin D6. Then the AD3 and the WaveGen Software was utilized to generate a square wave that was connected to D6 as well.



Using the AD3 to measure both the WaveGen generated waveform and the output pin D13, we increased the frequency of the input square wave to 60kHz which was the point of the ISR missing pulses. At this frequency the Arduino can no longer keep up with the rate of the interrupts.



4.B.3 Comparators as Level Shifters

The Motor encoders output at 8V which exceeds the Arduino input level of 5V. To safely interact with the Arduino, we utilized a comparator circuit as a level shifter to convert encoder pulses to be in the range of 5V digital signals. Using pins D2 and D4 to take the output of the level shifter circuit we were able to adjust the arduino code so each side can act independently.

To determine the number of CPU cycles available while running at full speed we used two interrupt lines for each encoder (two total). We are cycling at 66Khz and our code tells us we get

around 3500 cycles per each encoder when running for two seconds. To find the CPU cycles we divide the arduino clock speed by the frequency in which the CPU skips. Which is 16 Mhz divided by 6kHz, giving us 266 clock cycles.

4.B.4 Read Encoder with Arduino

Our group devised an experiment to check the correct counting of pulses. We had the robot go forward for a specified number of time and measured the distance it traveled. In the arduino code, it would print the pulse count of each side after it was finished moving forward. Then we divided the distance by the number of pulses. We compared the left and right sides to ensure they were printing similar values. We then repeated the experiment five times to get a reliable average value. The counts were consistent and similar between both sides, ensuring that the ISRs were working as planned.

4.B.5

Using findings from 4B4, the group implemented an arduino program to move the robot based on the number of encoder pulses. Using a ruler, we measured out two feet and adjusted the number of pulses until the robot moved that distance. For testing the robots ability to spin. We had the robot turn both counter clockwise and clockwise by having one side's wheels spin forward and the other backwards. We made adjustments to the turn pulses until the robot could reliably spin 180 and 360 degrees. Having these measurements, one can take these pulses and utilize them for more advanced routes, which we did later in the lab.

4.B.6

The control system utilizing pulses from this experiment resulted in incredible accuracy and repeatability. The robot consistently returned to its starting position with little to no deviation from its path. In comparison to Experiment 4A9, which relied on time instead of encoder pulses the results were similar when the code was configured for a specific surface. The advantage to the encoder feedback was it executed the intended code on any surface I tried, (floor, table, and carpet) despite all having different friction coefficients.

```
//Pin Definition
const int pinON = 6;

const int pinLeftForward = 11;
const int pinLeftBackward = 12;
const int pinLeftPWM = 10;

const int pinRightForward = 7;
const int pinRightBackward = 8;
const int pinRightPWM = 9;

const int leftEncoderPin = 2;
const int rightEncoderPin = 4;

//Initializing Left and Right Counts
volatile long leftCount = 0;
volatile long rightCount = 0;

//Pulse Params
const int targetPulsesForward = 2500; //2ft Forward
const int targetPulsesTurn = 1400; //180° spin

void setup() {
 pinMode(pinON, INPUT_PULLUP);

 pinMode(pinLeftForward, OUTPUT);
 pinMode(pinLeftBackward, OUTPUT);
 pinMode(pinLeftPWM, OUTPUT);

 pinMode(pinRightForward, OUTPUT);
 pinMode(pinRightBackward, OUTPUT);
 pinMode(pinRightPWM, OUTPUT);

 pinMode(13, OUTPUT);
 digitalWrite(13, LOW);

 //Encoder pins
 pinMode(leftEncoderPin, INPUT);
 pinMode(rightEncoderPin, INPUT);

 attachInterrupt(digitalPinToInterrupt(leftEncoderPin), leftEncoderISR, RISING);
 attachInterrupt(digitalPinToInterrupt(rightEncoderPin), rightEncoderISR, RISING);
```

```
 analogWrite(pinLeftPWM, 200);
 analogWrite(pinRightPWM, 200);

 Serial.begin(9600);
}

void loop() {
 // Waiting for button press
 while (digitalRead(pinON) == HIGH);
 digitalWrite(13, HIGH);
 delay(1000);

 // Forward 2 ft
 resetEncoders();
 moveForward(targetPulsesForward);
 stopMotors();
 delay(1000);

 // Clockwise turn
 resetEncoders();
 turnClockwise(targetPulsesTurn);
 stopMotors();
 delay(1000);
 // Forward 2 ft again
 resetEncoders();
 moveForward(targetPulsesForward);
 stopMotors();
 delay(1000);

 // Counter-clockwise turn
 resetEncoders();
 turnCounterClockwise(targetPulsesTurn);
 stopMotors();
 delay(1000);
 }

//ISR Functions --> Incrementing Left and Right
void leftEncoderISR() {
 leftCount++;
}
```

```
void rightEncoderISR() {
 rightCount++;
}

//Reset Functions
void resetEncoders() {
 noInterrupts();
 leftCount = 0;
 rightCount = 0;
 interrupts();
}

void stopMotors() {
 digitalWrite(pinLeftForward, LOW);
 digitalWrite(pinLeftBackward, LOW);
 digitalWrite(pinRightForward, LOW);
 digitalWrite(pinRightBackward, LOW);
}

//Movement Functions
void moveForward(int targetPulses) {
 digitalWrite(pinLeftForward, HIGH);
 digitalWrite(pinRightForward, HIGH);

 while (leftCount < targetPulses && rightCount < targetPulses) {
 }
}

void turnClockwise(int targetPulses) {
 digitalWrite(pinLeftForward, HIGH);
 digitalWrite(pinRightBackward, HIGH);

 while (leftCount < targetPulses && rightCount < targetPulses) {
 }
}

void turnCounterClockwise(int targetPulses) {
 digitalWrite(pinLeftBackward, HIGH);
 digitalWrite(pinRightForward, HIGH);

 while (leftCount < targetPulses && rightCount < targetPulses - 150) {
 }
```

```
}
```

Conclusion:

In experiments 4A and 4B, we progressed from mastering open-loop speed control to achieving precise closed-loop position control on our four-wheeled Arduino Nano Every robot. In 4A, we built and tuned dual speed-sensor circuits, designed PWM-Analog filters for stable reference voltages, calibrated feedback loops for matched motor speeds, implemented H-bridge direction control, and verified movement repeatability with programmed drive-and-turn sequences. In 4B, we characterized the microcontroller's interrupt latency, used comparators to level-shift 8 volt encoder outputs into reliable logic pulses, wrote interrupt routines to count those pulses, and developed PWM-ramp algorithms that delivered accurate linear and rotational moves. Our key insight is that dependable robotic motion demands both careful analog signal conditioning and robust, interrupt driven firmware. We learned the importance of simulating and scope-checking each stage before hardware assembly, and of keeping wiring and code organized to streamline debugging. Our primary limitation was real-world non-idealities - component tolerances, filter ripple, and jitter - which introduced small but cumulative errors. Providing example waveform traces in future labs would help us quickly verify correct operation in the future.